

Средства симуляции ЦП и ОС

Лекция №5

Державин Андрей
Шурыгин Антон

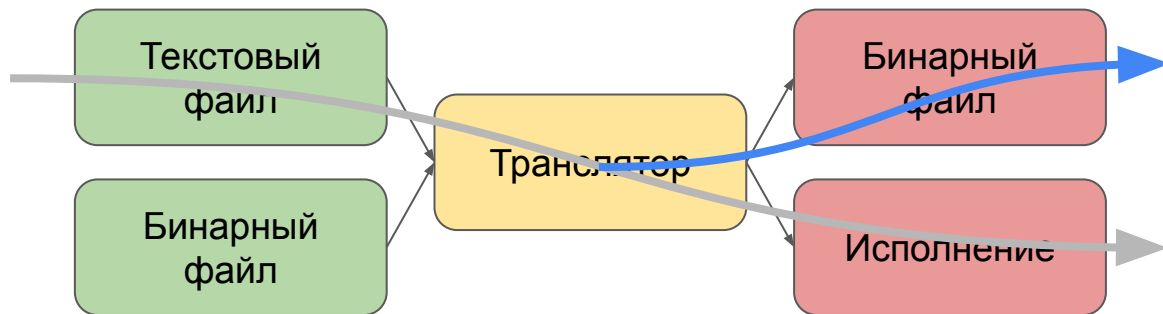
Двоичная трансляция

Вспомним былое

- Как работает интерпретатор?
- Исполняет инструкции по очереди, одна за другой
- Какие преимущества и недостатки интерпретирующего симулятора?
- + Простота реализации
- + Простота модификации
- Низкая скорость работы (даже с оптимизациями)
 - Как вы думаете, из-за чего?

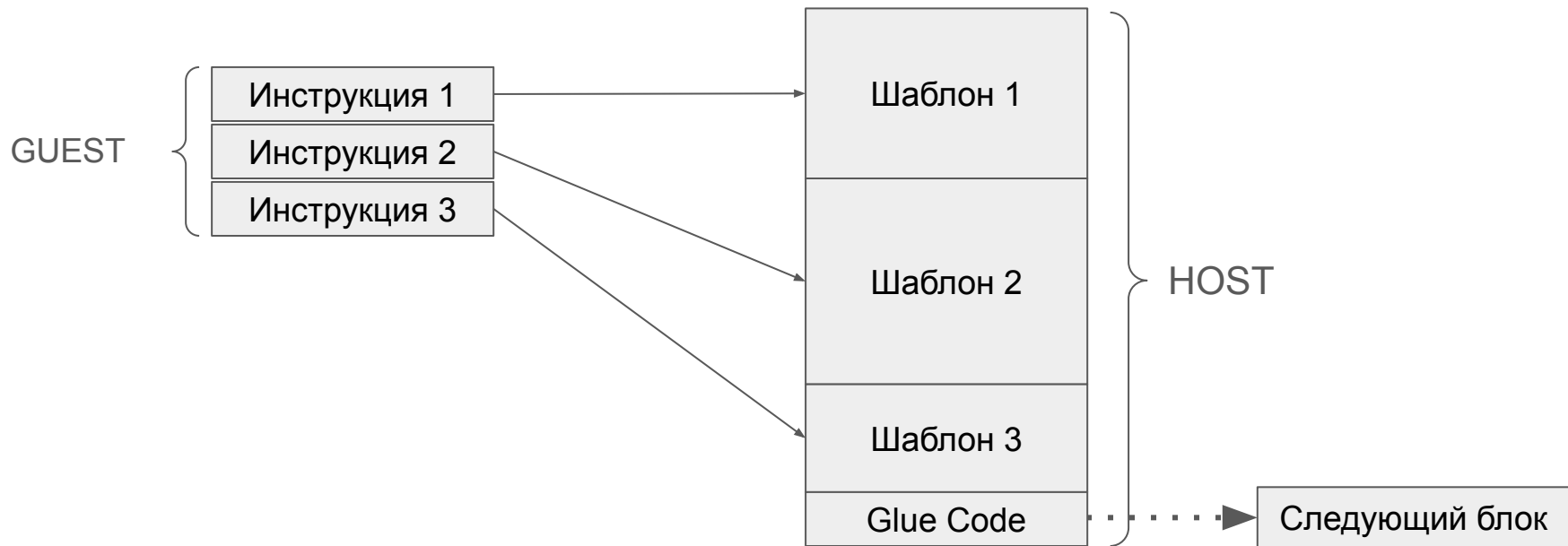
Двоичная трансляция

- Интерпретатор - простой, но медленный
- Как мы можем ускорить работу симулятора?
 - Вспомним аналогию с языками программирования
 - Идея: “Компилировать” блоки кода гостевой архитектуры



Как компилировать?

- Интерпретатор - простой, но медленный
- Идея: “Компилировать” блоки кода гостевой архитектуры



Шаблоны

- Рассмотрим шаблон для инструкции add (RV32I)
- Регистр rdi хранит адрес начала CpuState (регистровый файл)
- Можно ли как-то обобщить шаблон?

```
add x1, x2, x3
```



```
mov edx, DWORD PTR [rdi+0x8]  
add edx, DWORD PTR [rdi+0xC]  
mov DWORD PTR [rdi+0x4], edx
```

Шаблоны

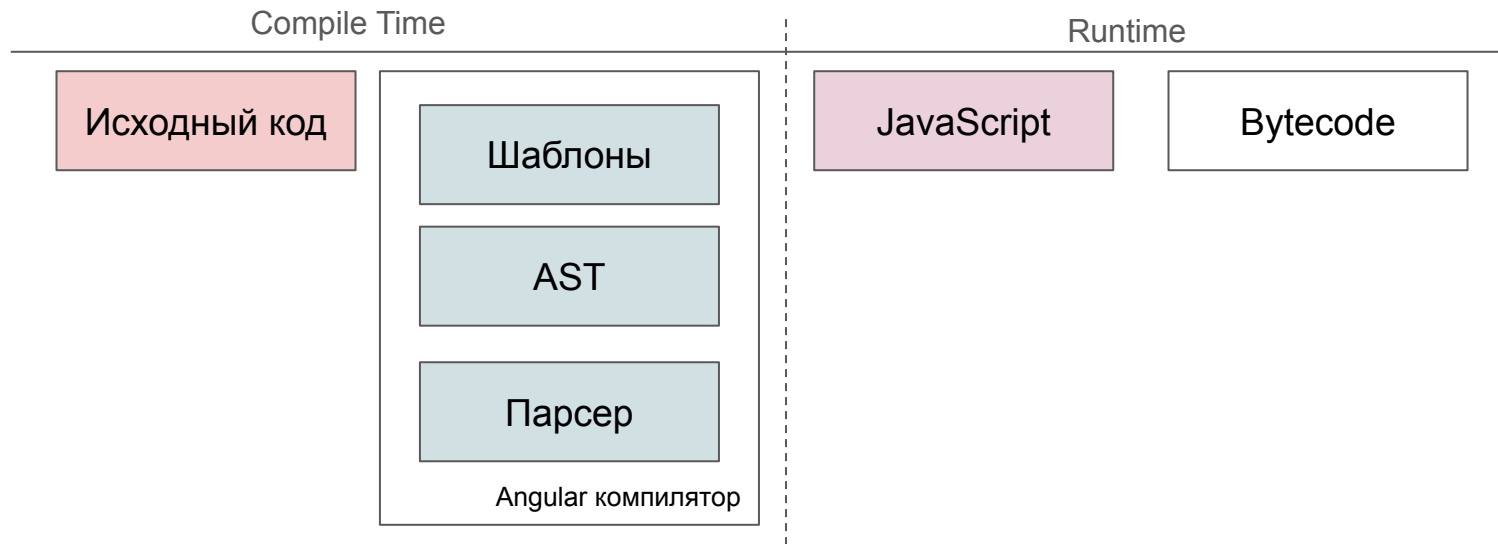
- Рассмотрим капсулу для инструкции add (RV32I)
- Регистр rdi хранит адрес начала `CpuState` (регистровый файл)
- Храним “шаблон” капсулы для каждой инструкции

```
add rd, r1, r2
```

```
mov edx, DWORD PTR [rdi+OF_R1]  
add edx, DWORD PTR [rdi+OF_R2]  
mov DWORD PTR [rdi+OF_RD], edx
```

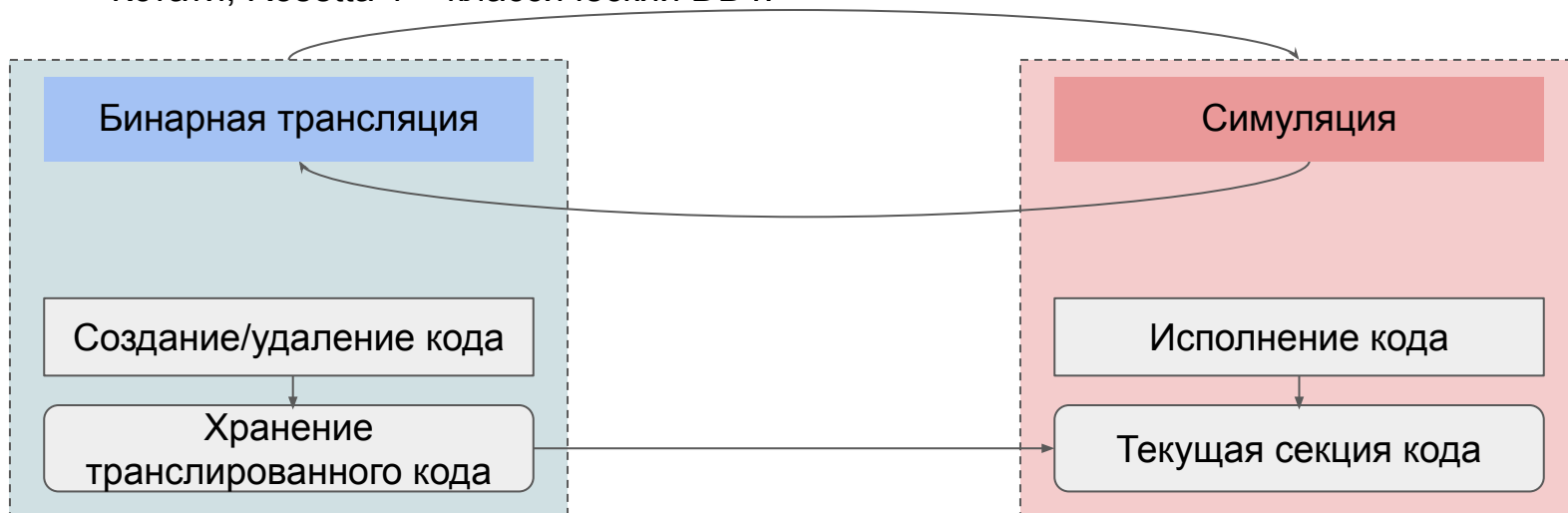
Типы трансляции

- Статическая бинарная трансляция – трансляция всей программы из гостевого кода в хозяйский до ее запуска (AOT-компиляция)
 - [Running 32-Bit x86 Applications on Alpha NT](#)
 - Apple Rosetta 2 (поддержка AOT-компиляции при установке приложения)



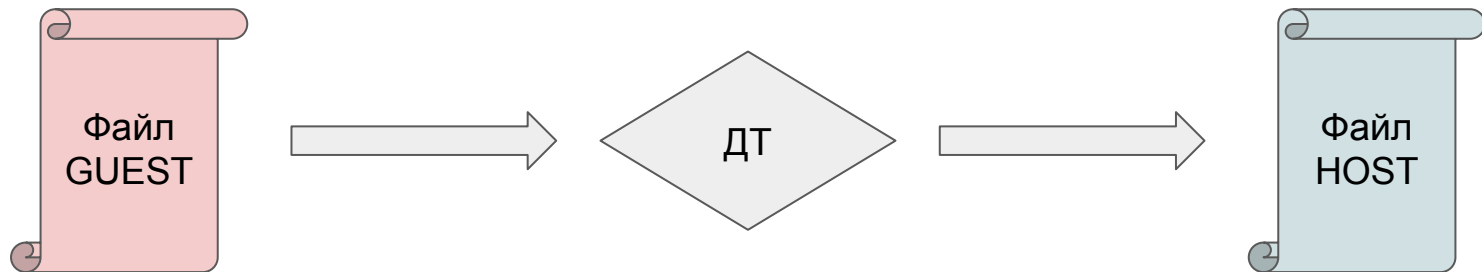
Типы трансляции

- Динамическая бинарная трансляция – трансляция во время исполнения программы по мере необходимости.
 - Чаще применяется для задач симуляции
 - QEMU
 - Кстати, Rosetta 1 – классический DDT.



Двоичная трансляция

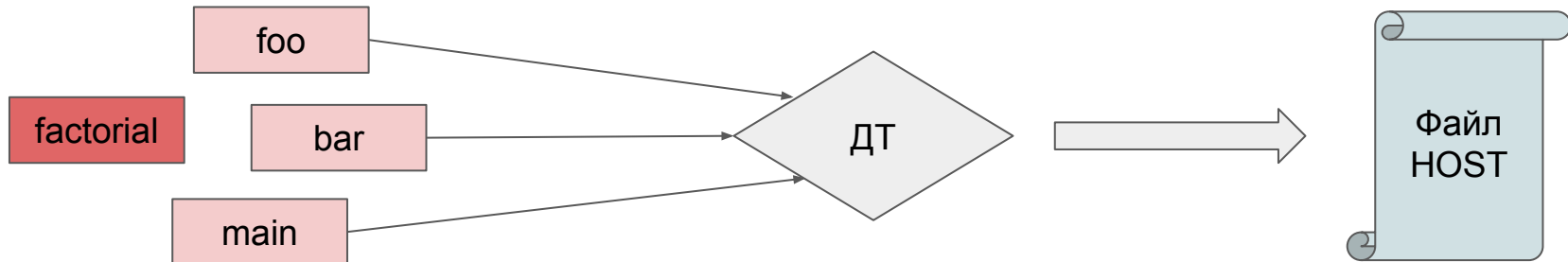
- Как определим единицу бинарной трансляции?
- Предложение №1: файл/модуль



- Неэффективное использование ресурсов
- Нет доступа к runtime информации
- + Статически преобразованный образ можно запускать неограниченное число раз
- + Нативное исполнение

Двоичная трансляция

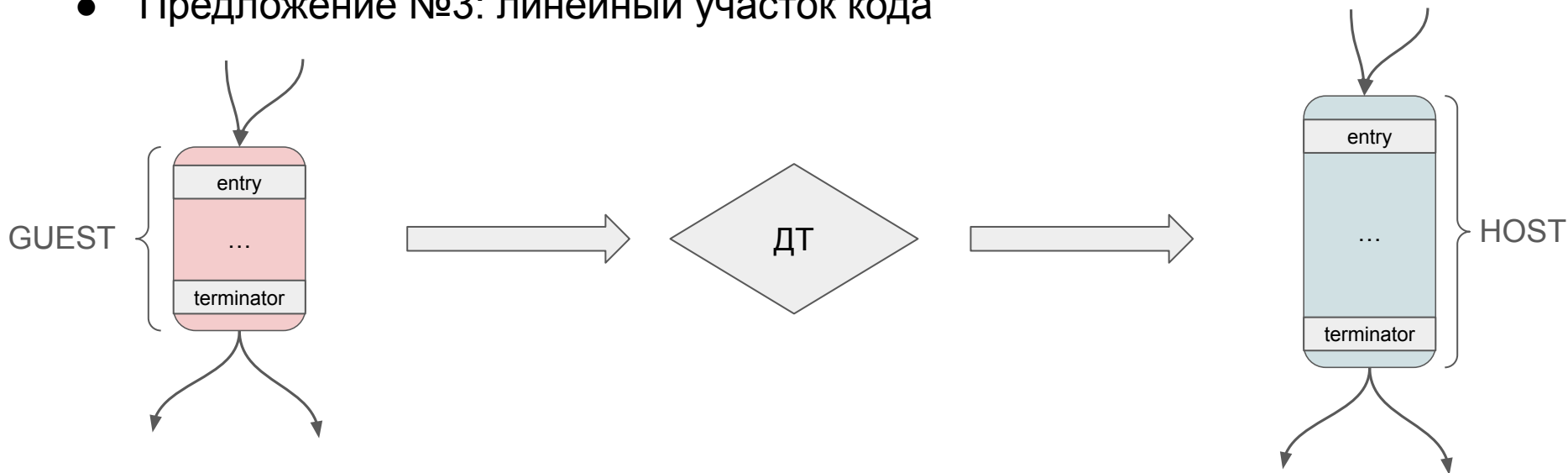
- Как определим единицу бинарной трансляции?
- Предложение №2: функция/процедура



- Дополнительная память под кэш трансляций
- Межпроцедурные оптимизации облагаются дополнительным анализом
- + Функции транслируются по мере загрузки библиотеки
- + Самая интуитивно понятная разработчику единица

Двоичная трансляция

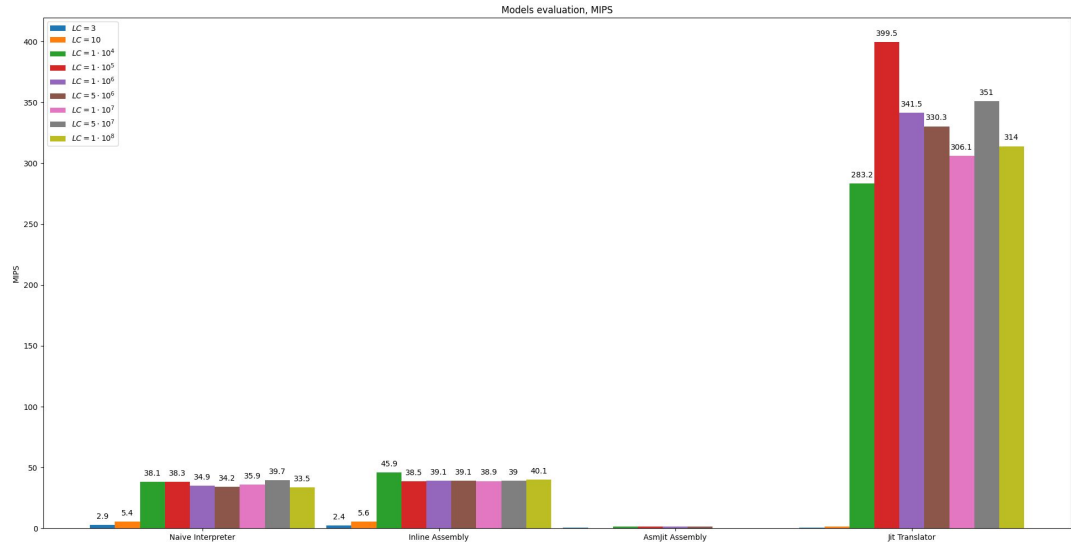
- Как определим единицу бинарной трансляции?
- Предложение №3: линейный участок кода



- Дополнительная память под кэш трансляций
- Высокие накладные расходы при частом переключении транслятор/симулятор
- + Быстрый старт, поскольку трансляция происходит по требованию

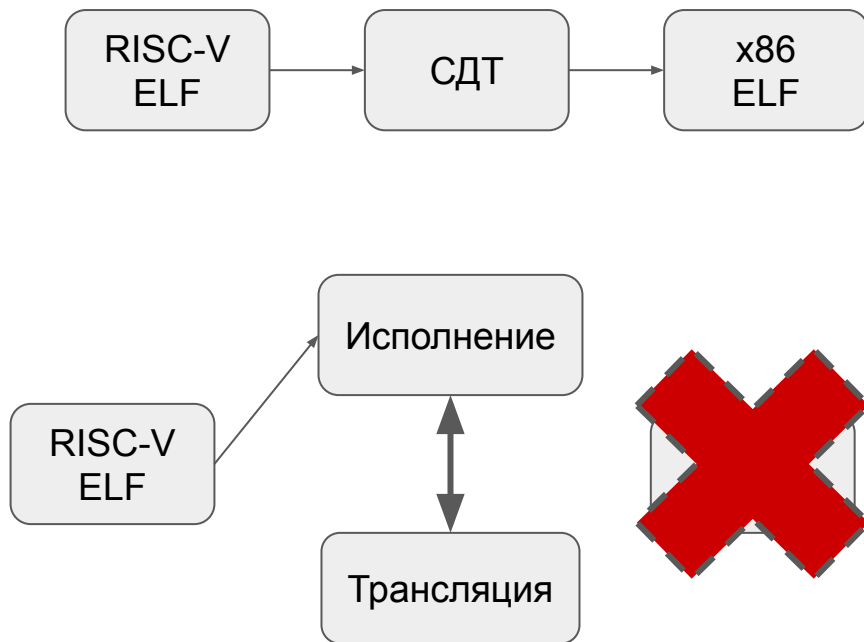
Давайте проверим?

- Возьмем наивный интерпретатор и гибридный JIT-транслятор
- Напишем простенькую программу на TOY ISA
- Сравним скорость исполнения



Двоичная трансляция

- Таким образом, получается:
- Статическая трансляция
 - “Переводит” ELF на другую СК
 - Обнаружение кода
 - Сложность интроспекции
- Динамическая трансляция
 - Трансляция исполняемого кода по запросу
 - Ограниченность оптимизаций



Серебряная пуля?

- Разобрали метод ускорения интерпретатора
- Транслируем на лету в хозяйский код
- Исполняется кратно быстрее
- Все хорошо
- Но есть одна проблема..

Проблема SMC

Понятие SMC

- Обычно исполняемый код и обрабатываемые данные находятся в одной физической памяти
- Создание программ, которые в процессе работы изменяют код других программ, в т.ч. свой собственный
- Такое явление получило название самомодифицирующийся код (англ. self-modifying code, SMC)

Проблема SMC

- СДТ не может поддерживать программы с SMC
- Необходимо заранее знать весь исполняемый код
- Кодировка SMC инструкции может быть результатом вычислений

Проблема SMC

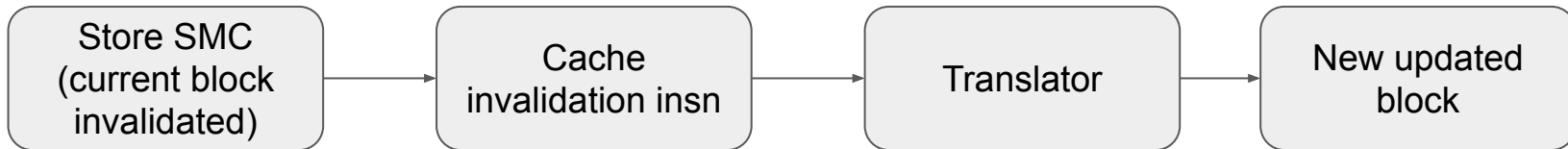
- Гостевой код изменяется при исполнении SMC программы.
- Проблема: возникает вероятность, что уже оттранслированные блоки кода провисли и перестали соответствовать содержимому памяти модели.

Проблема SMC

- Гостевой код изменяется при исполнении SMC программы.
- ✓ Проблема: возникает вероятность, что уже оттранслированные блоки кода провисли и перестали соответствовать содержимому памяти модели.
- Решение – проверка всех записей в память для дальнейшего сброса состояния модели и повторной трансляции затронутых блоков.
- Снова проблема – долгий процесс бинарной трансляции одного блока:
 - Скорость работы симулятора снижается
 - Простой интерпретатор оказывается более производительным

Проблема SMC

- Что делать, если инструкция меняет другую инструкцию в том же блоке трансляции?
- Precise SMC
- Нужно как то выйти из середины блока в транслятор
- Такая проблема есть только на x86 guest
- ARM, RISC-V, ... требуют исполнения инструкций которые инвалидируют кэш инструкций (даже если его нет!!)
- QEMU: разбивать блоки на таких инструкциях



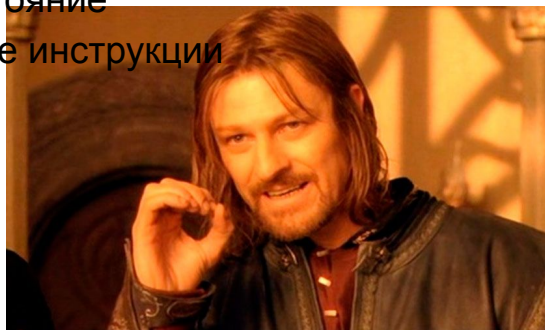
Прямое исполнение

А что если HOST = GUEST?

- Важный случай бинарной трансляции – совпадение гостевой и хозяйской архитектуры
- Возникает желание (и возможность!) скопировать гостевой код как хозяйский или даже *исполнить его напрямую*
- Подобные режимы симуляции называются **прямое исполнение** (англ. direct execution, DEX)

Прямое исполнение

- Хозяйская архитектура совпадает с гостевой
 - Что может пойти не так?
- В любом случае необходимо обеспечить изоляцию исполнения кода гостевых приложений от кода симулятора
- Примеры операций, нарушающих условия изолированности?
 - Доступ к памяти и периферии
 - Архитектурное состояние
 - Привилегированные инструкции

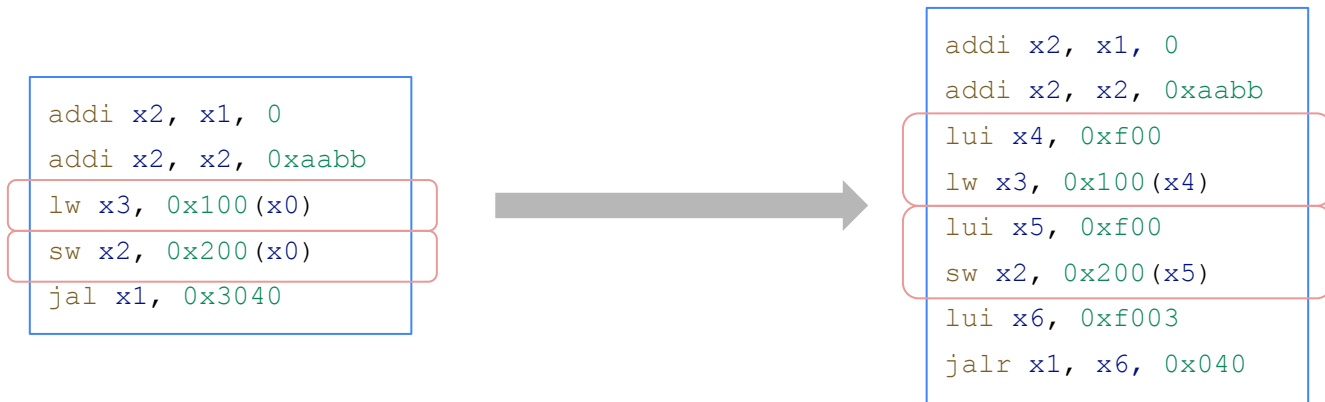


Прямое исполнение

- Какие у вас есть идеи, как можно обойти все эти уязвимости?
- Предпросмотр кода – процесс инспектирования гостевого кода на предмет инструкций, запрещенных к ***прямому исполнению***.
- В результате предпросмотра проблемные инструкции мы можем “обезвредить”:
 - Заплата (англ. patch)
 - Заглушка (англ. stub)

Прямое исполнение

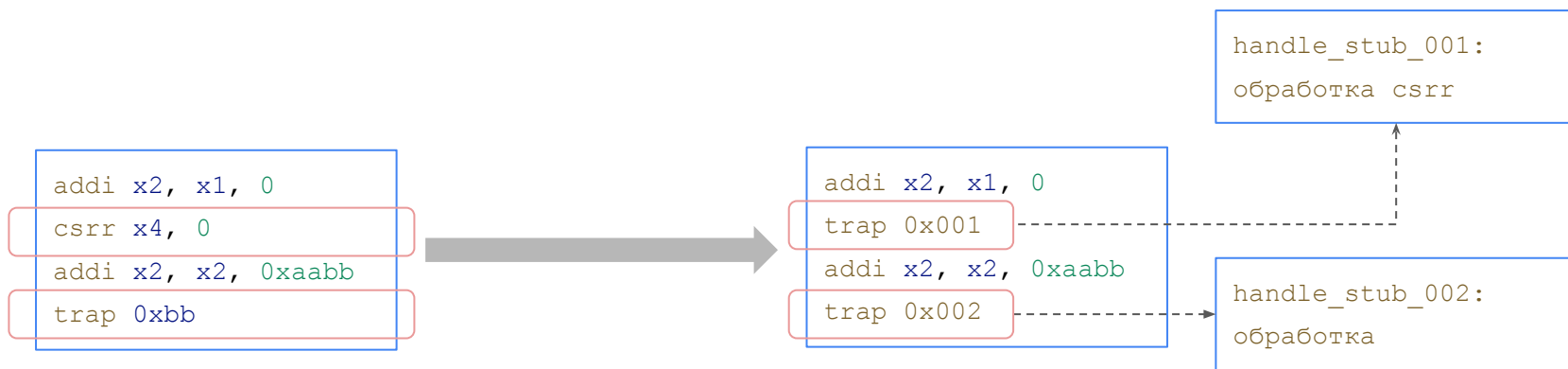
- Заплата (англ. patch) — фрагмент машинного кода, заменяющий одну исходную инструкцию



После применения заплаток все адреса обращений к памяти заменены ссылками на симулируемые

Прямое исполнение

- Заглушка (англ. stub) — приём, в котором исходная инструкция заменяется на другую, вызывающую процедуру, подготовленную симулятором



Инструкции, требующие контролируемого выполнения, заменяются на инструкции безусловной передачи управления в процедуру-обработчик симулятора

Сложности DEX

- Строго говоря, спектр возможных проблем при DEX широкий
 - Переменная длина инструкций
 - SMC
 - Значение гостевых регистров может быть недоступно
 - и так далее...

Оптимизирующая трансляция

Оптимизирующая трансляция

- Бинарная трансляция приносит не только сложности и проблемы в работе модели
- Мы обрели механизм преобразовать блок трансляции так, чтобы он работал быстрее

Оптимизирующая трансляция

- Бинарная трансляция приносит не только сложности и проблемы в работе модели
- Ведь у нас появляется механизм преобразовать блок трансляции так, чтобы он работал быстрее
- Какие компиляторные оптимизации вы знаете?
 - Удаление мертвого кода (англ. dead code elimination)
 - Удаление общих подвыражений (англ. common subexpression elimination)
 - Свертка констант (англ. constant folding)
 - Дублирование констант (англ. constant propagation)
 - Глазок (англ. peephole)



Оптимизирующая трансляция

- Рассмотрим пример часто используемой оптимизации ([Helmstetter et al., 2011](#))



Шаблонная трансляция

Вспомним еще раз

- Динамический двоичный транслятор
- Преобразует код гостевой архитектуры в код на хозяйской “на лету”
- Основная цель - повышение производительности в сравнении с интерпретатором
- Ключевые этапы:
 - Обнаружение горячих блоков
 - Трансляция
 - Оптимизация
 - Кэширование трансляций

А чем ДДТ не устраивает?

- Одна трансляция кратно дольше одного такта интерпретатора
 - Почему?
- Скорость трансляции критична
- Узкий спектр доступных оптимизаций
- Не высокое качество получаемого кода



ДДТ++

- ДДТ транслирует в рантайме
 - Хочется тратить как можно меньше времени на трансляцию
- Храним скомпилированные шаблоны для инструкций на этапе компиляции
- Как быть с операндами инструкции?
 - Запомним смещения в шаблоне для каждого значения операндов

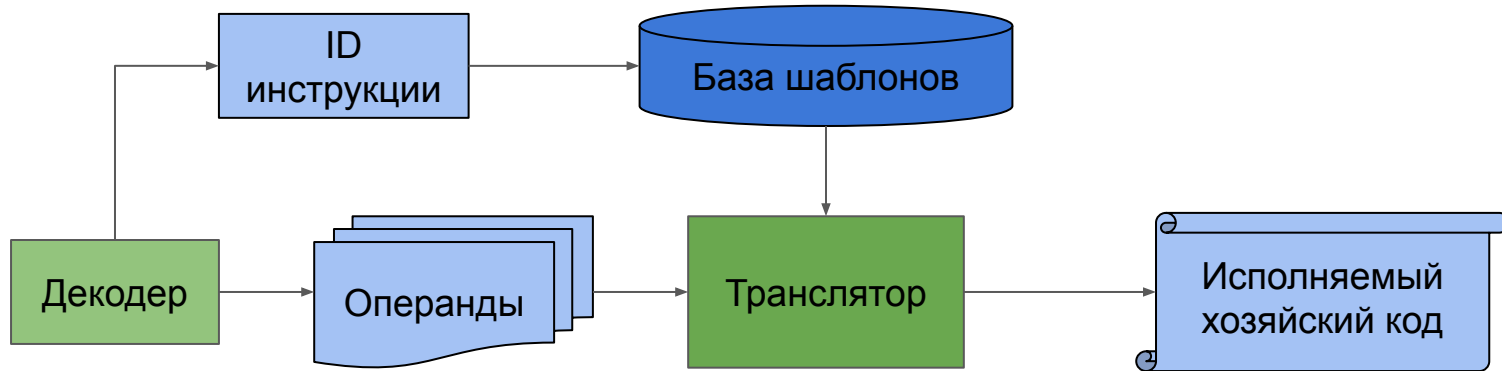
```
add rd, r1, r2
```



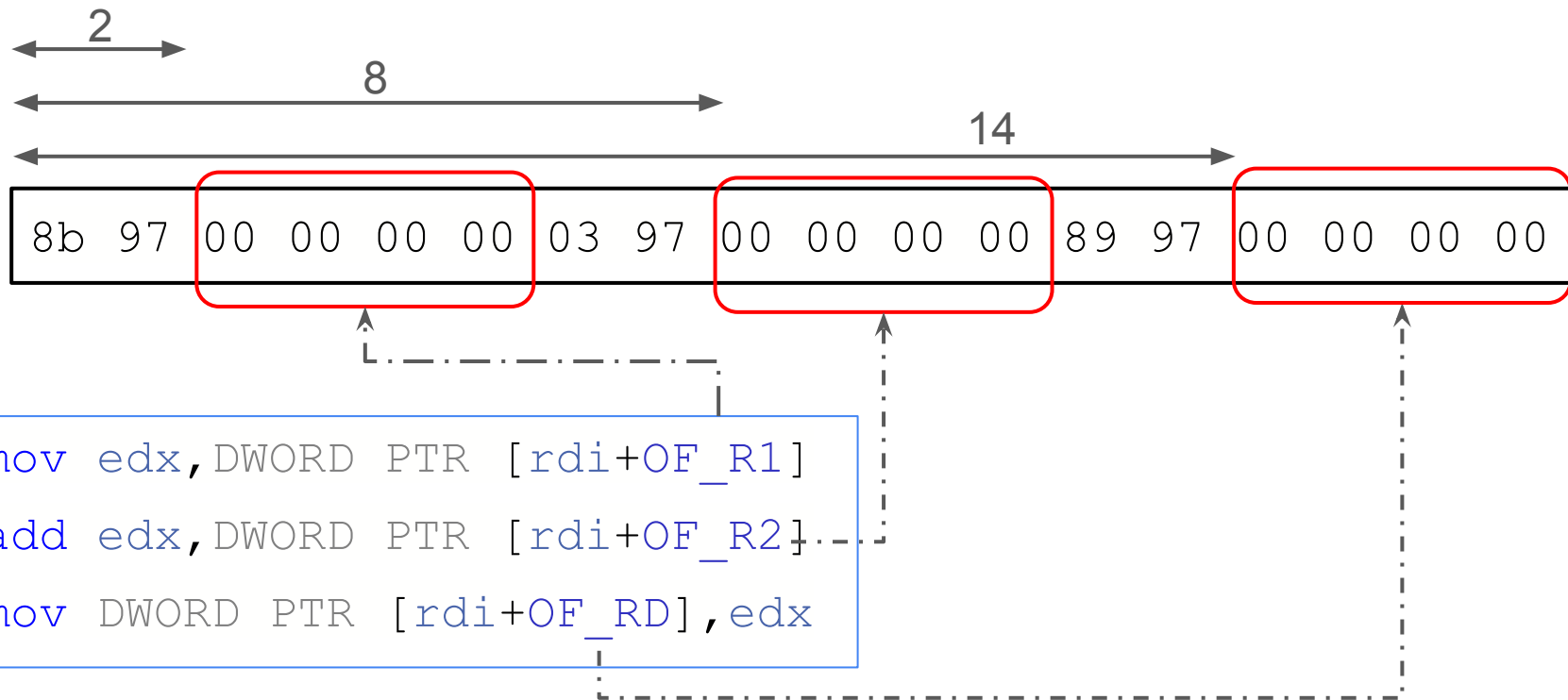
```
mov edx, DWORD PTR [rdi+OF_R1]  
add edx, DWORD PTR [rdi+OF_R2]  
mov DWORD PTR [rdi+OF_RD], edx
```

Шаблонная трансляция

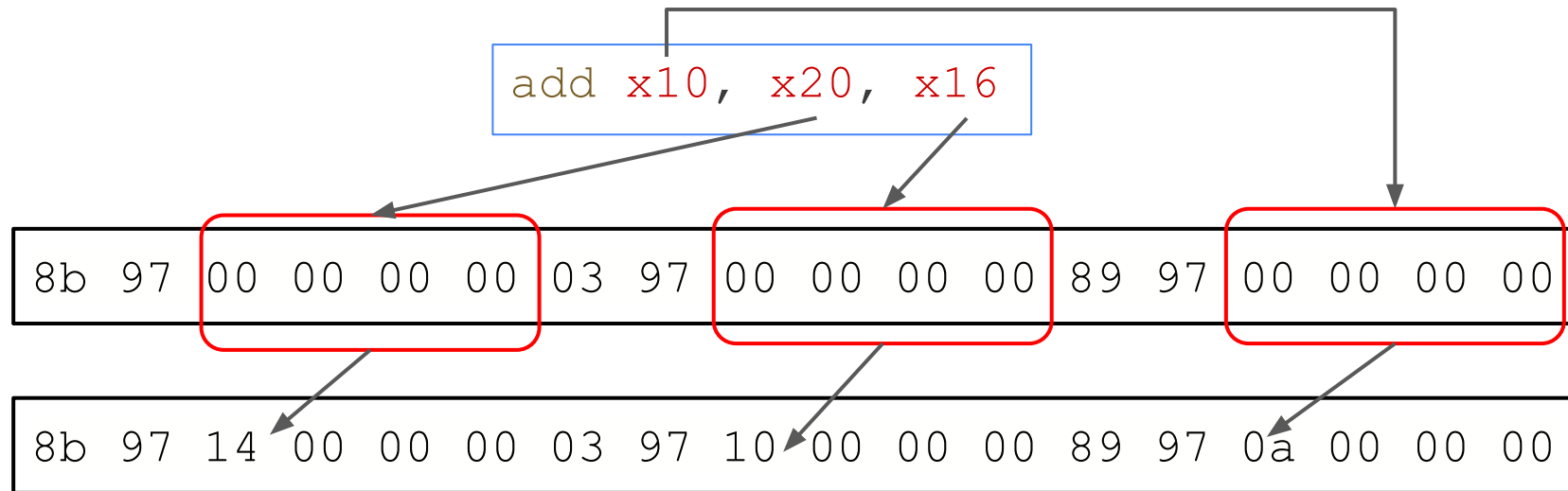
- Шаблоны формируются до сборки транслятора
- Аргументы шаблона подставляются значениями операндов в рантайме
- Готовый код выполняется на хозяйской машине



Подготовка шаблона



Подстановка шаблона



```
mov edx, DWORD PTR [rdi+20]
add edx, DWORD PTR [rdi+16]
mov DWORD PTR [rdi+10], edx
```

Подстановка шаблонов. Факторизация

- Шаблон реализован в общем виде
- Некоторые случаи специальных операндов (напр. XZR) могут существенно упростить код шаблона
- Реализуем специальные версии шаблонов под такие случаи
- Транслятор подбирает лучший вариант
- Может привести к комбинаторному взрыву

```
add x2, x0, x1
```



```
mov edx, DWORD PTR [rdi+OF_R2]  
mov DWORD PTR [rdi+OF_RD], edx
```


Шаблонная трансляция. Итоги

- Позволяет использовать любые оптимизации для шаблонов
 - Не влияет на скорость трансляции
- Трансляция почти мгновенная
- Ручная реализация шаблонов под каждую хозяйскую архитектуру усложняет разработку
- Требуется большое число шаблонов
 - Как минимум один для каждой возможной инструкции
- Факторизациякратно увеличивает число шаблонов

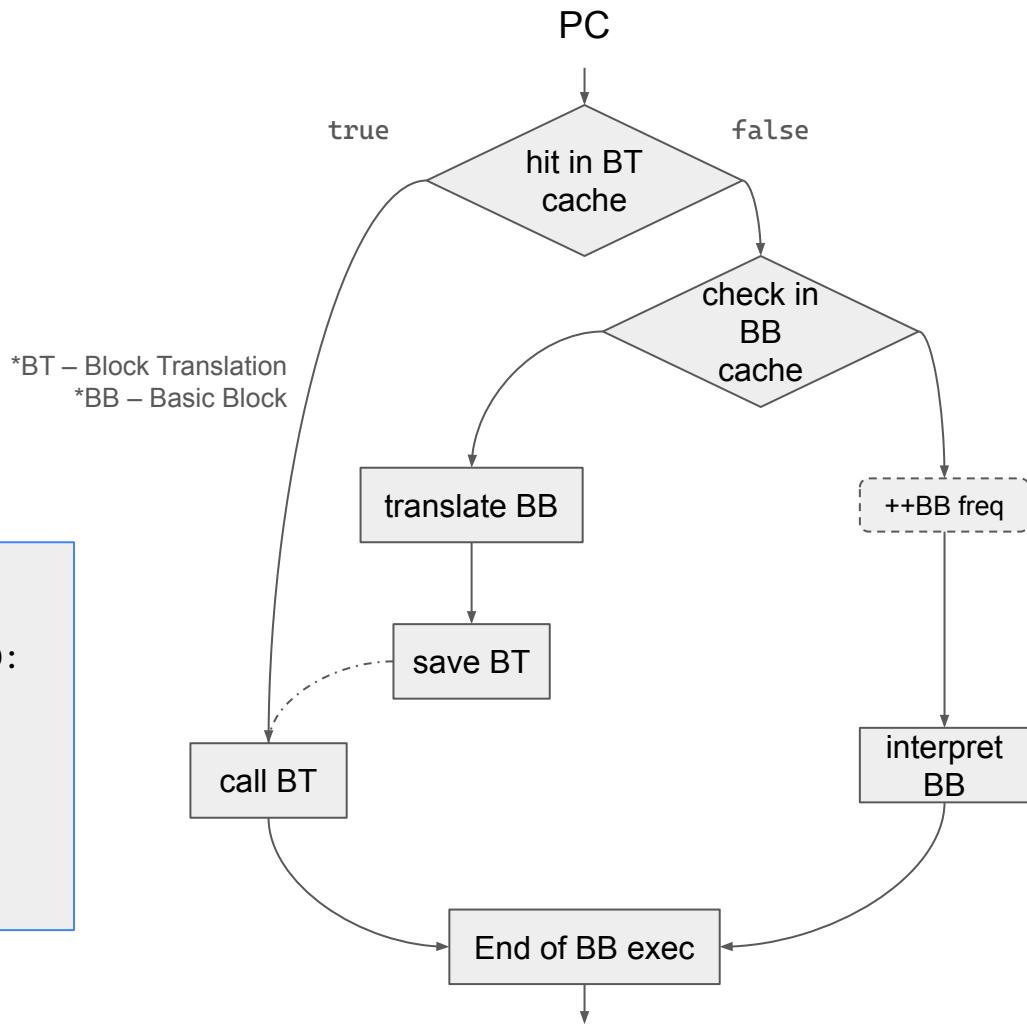
ДЗ-1*

JIT

- Рассмотрим упрощенный алгоритм симуляции с JIT-трансляцией
- В рамках выполнения ДЗ-1 можно положить **THRESHOLD = 0**
- Упрощенная схема взята из [статьи](#)

```
if PC in BTCache:  
    BTCache[PC].call()  
else if BBCache[PC].exec_count ≥ THRESHOLD:  
    code = translate(PC)  
    BTCache[PC] = code  
    code.call()  
else:  
    BBCache[PC].exec_count++  
    interpret(PC)
```

Листинг. 1. Алгоритм симуляции с JIT-трансляциями



asmjit

- Библиотека для генерации машинного кода
- Вызов метода добавляет байты инструкции в хранилище кода
- Доступные хозяйские СК:
 - arm
 - x86
- Реализованы контейнеры для хранения исполняемого кода

```
1  case isa::Opcode::kAdd: {
2      cc.mov(temp, toDwordPtr(cpu.regs[insn.src1]));
3      cc.mov(temp2, toDwordPtr(cpu.regs[insn.src2]));
4      cc.add(temp, temp2);
5      //
6      cc.mov(
7          asmjit::x86::dword_ptr((size_t)&(cpu.regs[insn.dst])),
8          temp);
9
10     cc.mov(temp, toDwordPtr(cpu.pc));
11     cc.add(temp, 1);
12
13     cc.mov(asmjit::x86::dword_ptr((size_t)&(cpu.pc)), temp);
14
15     break;
16 }
```

Шаблон инструкции ADD с использованием библиотеки asmjit